TP02

AUTOMATIZACIÓN DE PRUEBAS Y PIPELINE CI/CD

Implementación de Testing Automatizado con Python y GitHub Actions

DESARROLLO → GITHUB → CI/CD PIPELINE → TESTS → RESULTADOS

ALUMNO:	Matías Solivella
LEGAJO:	1197376
MATERIA:	Testing de Aplicaciones
DOCENTE:	Laime Huanca, Abel Israel
FECHA:	24/05/2025

UADE
UNIVERSIDAD ARGENTINA DE LA EMPRESA

Ingeniería en Informática

Contenido

Introducción **2**
Explicación paso a paso del trabajo **2**
Parte 1 – Diseño del escenario de prueba 2
Parte 2 – Automatización de las pruebas 2
Estructura final del proyecto 2
Código de la funcionalidad 3
Implementación de los tests 3
Explicación de los casos automatizados 3
Ejecución de pruebas 4
Dependencias 4
Parte 3 – Implementación del pipeline (CI/CD) 4
Configuración del pipeline 4
Parte 4 – Evidencia y análisis 5
Evidencia del pipeline 5
Casos de prueba 5
¿Qué ventaja aporta automatizar pruebas dentro del pipeline? 6
¿Qué impacto tiene esto en la calidad del software? 6
Parte 5 – Reflexión 7
¿Qué beneficios tiene automatizar pruebas frente a hacerlas manualmente? 7
¿Qué dificultades encontraste al implementar el pipeline? 7
¿Dónde aplicarías este enfoque en un entorno real? 7
Conclusiones personales **7**
Propuestas de futuras mejoras **8**
Bibliografía **8**

Introducción

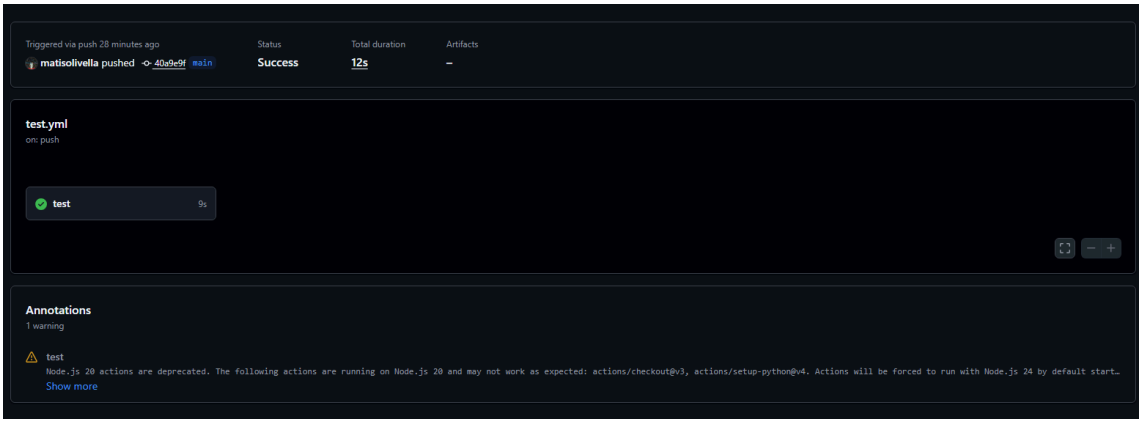
El presente trabajo tiene como objetivo implementar un flujo básico de automatización de pruebas integrado a un pipeline de integración continua (CI/CD), utilizando Python y GitHub Actions.
Se busca aplicar conceptos de testing y DevOps para validar el correcto funcionamiento de una funcionalidad simple, asegurando calidad mediante pruebas automatizadas ejecutadas de forma continua.

Explicación paso a paso del trabajo

Parte 1 – Diseño del escenario de prueba

Se desarrolló una funcionalidad simple de una calculadora que permite realizar la operación de división entre dos números.

El sistema debe permitir dividir dos números y devolver el resultado correcto cuando los valores son válidos.
En caso de intentar dividir por cero, el sistema debe manejar correctamente el error mediante una excepción.
Este escenario permite validar tanto el comportamiento esperado como la gestión de errores y casos límite.



Parte 2 – Automatización de las pruebas

Se utilizó **pytest**, una librería de Python que permite crear y ejecutar pruebas de manera simple, clara y eficiente.

Estructura final del proyecto

tpo2-testing-ci-cd/

|

```
├── app/
│   ├── __init__.py
│   └── calculadora.py
│
├── tests/
│   └── test_calculadora.py
│
├── main.py
├── requirements.txt
└── .github/
    └── workflows/
        └── test.yml
```

Código de la funcionalidad

```python
def dividir(a, b):
    if b == 0:
        raise ValueError("No se puede dividir por cero")
    return a / b
```

Implementación de los tests

```python
import pytest
from app.calculadora import dividir

def test_ok():
    assert dividir(10, 2) == 5

def test_error():
    with pytest.raises(ValueError):
        dividir(10, 0)

def test_edge():
    assert dividir(0, 5) == 0
```

Explicación de los casos automatizados

Se implementaron tres pruebas automatizadas que cubren distintos escenarios del sistema:

- **Caso exitoso:** valida que la operación de división funcione correctamente con valores válidos.
- **Caso de error:** verifica que el sistema gestione correctamente la división por cero mediante una excepción.
- **Caso borde:** evalúa el comportamiento del sistema cuando el numerador es cero.

Ejecución de pruebas

python -m pytest

Dependencias

pytest
pytest-html

Parte 3 – Implementación del pipeline (CI/CD)

Se utilizó **GitHub Actions**, una herramienta que permite automatizar procesos dentro del repositorio, como la ejecución de pruebas, instalación de dependencias y validación del código.

Configuración del pipeline

Archivo: .github/workflows/test.yml

.github > workflows > test.yml

```yaml
name: CI Pipeline

on:
  push:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - uses: actions/setup-python@v4
        with:
          python-version: '3.10'

      - run: python -m pip install --upgrade pip
      - run: python -m pip install pytest pytest-html

      - run: python -m pytest tests --html=report.html
```

El pipeline se ejecuta automáticamente cada vez que se realiza un **push** al repositorio.
Durante su ejecución realiza las siguientes tareas:

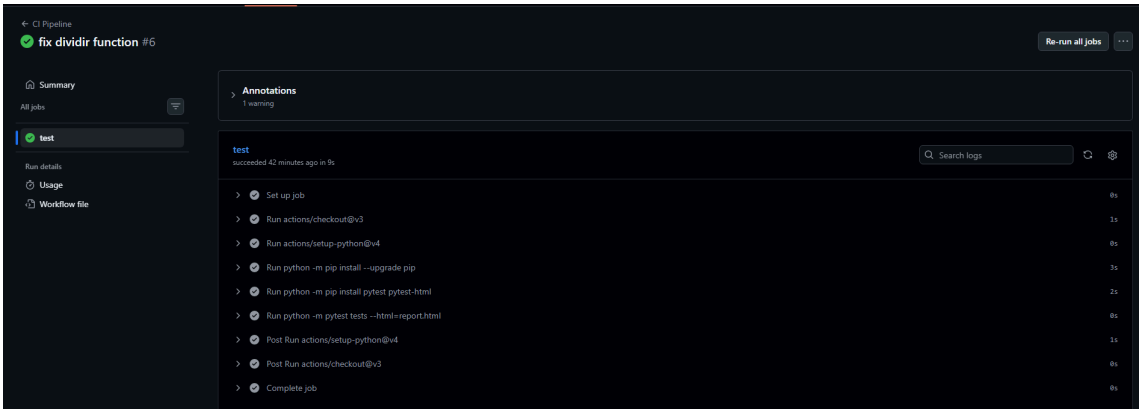
1. Descarga el código del repositorio
2. Configura el entorno de Python
3. Instala las dependencias necesarias
4. Ejecuta los tests automatizados
5. Genera un reporte en formato HTML
6. Guarda el reporte como artefacto descargable

Este proceso permite validar automáticamente el funcionamiento del sistema ante cada cambio, asegurando que no se introduzcan errores en el código.

Parte 4 – Evidencia y análisis

Evidencia del pipeline

A continuación, se presenta la ejecución del pipeline en GitHub Actions, donde se observa que todos los tests fueron ejecutados correctamente y finalizaron en estado exitoso.



Los resultados obtenidos demuestran que la lógica implementada en la calculadora funciona correctamente en distintos escenarios.
Además, se valida que el sistema maneja adecuadamente los errores y responde correctamente ante casos límite.

Casos de prueba

Caso 1 – Exitoso

- Entrada: 10 / 2
- Resultado esperado: 5

```
PS C:\Users\Mati\tpo2-testing-ci-cd> py main.py
=== Calculadora de División ===
Ingrese el primer número: 10
Ingrese el segundo número: 2
Resultado: 5.0
```

Caso 2 – Error

- Entrada: 10 / 0
- Resultado esperado: Se lanza una excepción (ValueError)

```
PS C:\Users\Mati\tpo2-testing-ci-cd> py main.py
=== Calculadora de División ===
Ingrese el primer número: 10
Ingrese el segundo número: 0
Error: No se puede dividir por cero
```

Caso 3 – Caso borde

- Entrada: 0 / 5
- Resultado esperado: 0

```
PS C:\Users\Mati\tpo2-testing-ci-cd> py main.py
=== Calculadora de División ===
Ingrese el primer número: 0
Ingrese el segundo número: 5
Resultado: 0.0
```

¿Qué ventaja aporta automatizar pruebas dentro del pipeline?

La automatización de pruebas dentro del pipeline permite ejecutar validaciones de forma automática cada vez que se realiza un cambio en el código.
Esto reduce la necesidad de pruebas manuales, acelera el proceso de desarrollo y permite detectar errores de manera temprana.

¿Qué impacto tiene esto en la calidad del software?

El impacto en la calidad del software es altamente positivo, ya que se asegura que cada cambio sea validado antes de ser integrado.
Esto reduce la probabilidad de errores en producción y mejora la confiabilidad del sistema.

Parte 5 – Reflexión

¿Qué beneficios tiene automatizar pruebas frente a hacerlas manualmente?

La automatización de pruebas presenta múltiples beneficios en comparación con las pruebas manuales.

En primer lugar, permite ejecutar pruebas de forma mucho más rápida, lo que reduce significativamente los tiempos de validación.

Además, las pruebas automatizadas son repetibles y consistentes, ya que siempre se ejecutan de la misma manera, evitando errores humanos.

También permiten aumentar la cobertura de testing, ya que se pueden ejecutar múltiples escenarios en poco tiempo.

Por último, facilitan la integración con pipelines de CI/CD, permitiendo validar automáticamente cada cambio en el código.

¿Qué dificultades encontraste al implementar el pipeline?

Durante la implementación del pipeline surgieron dificultades relacionadas con la configuración del entorno, instalación de dependencias, reconocimiento de los tests y errores en la configuración del archivo YAML.
Sin embargo, resolver estos problemas permitió comprender mejor el funcionamiento de la integración continua y la importancia de una correcta configuración del proyecto.

¿Dónde aplicarías este enfoque en un entorno real?

Este enfoque puede aplicarse en cualquier proyecto de desarrollo de software donde múltiples desarrolladores realicen cambios frecuentes.
Es especialmente útil en entornos ágiles, donde se realizan integraciones constantes y se requiere validar rápidamente el funcionamiento del sistema.
También es fundamental en aplicaciones críticas, donde la calidad del software es clave, ya que permite detectar errores antes de que lleguen a producción.

Conclusiones personales

A lo largo de este trabajo se logró implementar un flujo básico de automatización de pruebas integrado a un pipeline de CI/CD, utilizando herramientas actuales como Python, pytest y GitHub Actions.
Se validó el correcto funcionamiento de una funcionalidad mediante tests automatizados y su ejecución continua dentro de un pipeline.
Esta experiencia permitió comprender la importancia de la automatización en el proceso de desarrollo, destacando su impacto positivo en la calidad, eficiencia y confiabilidad del software.

Propuestas de futuras mejoras

Como mejora futura, se podrían incorporar más casos de prueba para aumentar la cobertura del sistema, incluyendo pruebas adicionales y escenarios más complejos.
También se podría integrar testing de integración o pruebas end-to-end para validar el comportamiento completo del sistema.
Además, se podría optimizar el pipeline agregando validaciones adicionales, como análisis de calidad de código o ejecución de pruebas en distintos entornos.

Bibliografía

- Material de la cátedra “Testing de Aplicaciones”
- Documentación oficial de pytest
- Documentación oficial de GitHub Actions